PARALLEL AND DISTRIBUTED COMPUTING · LECTURE 6

Message Passing & MPI

How processes with separate memories work together by sending and receiving messages.



LECTURE 6

Learning outcomes

By the end of this lecture, students will be able to:

1. Explain message passing and why distributed-memory systems need it.
2. Describe MPI program structure: rank, size and communicators.
3. Write a basic MPI program using MPI_Send() and MPI_Recv().
4. Distinguish blocking / non-blocking and synchronous / asynchronous communication.
5. Use the collectives MPI_Bcast, MPI_Scatter, MPI_Gather and MPI_Reduce.
6. Compare OpenMP and MPI, and explain hybrid programming.

WHERE WE ARE

Recap: the story so far

Lecture 3

Forms of parallel processing

SIMD and MIMD: many processors doing work at the same time.

Lecture 4a

Memory models, GPUs, hybrid

Shared vs distributed memory; MPI named as the tool for distributed memory.

Lecture 4b

OpenMP

#pragma omp parallel creates a team of threads sharing memory.

Today · Lecture 6

Message passing & MPI

How processes with separate memories cooperate.

Question to open with: OpenMP threads share memory — so what do we do when the processors are in **different machines**?

OVERVIEW

Eight parts, from idea to code

01
Message passing

02
Introduction to MPI

03
Basic MPI functions

04
Point-to-point communication

05
Blocking and non-blocking

06
Collective communication

07
MPI and parallel computation

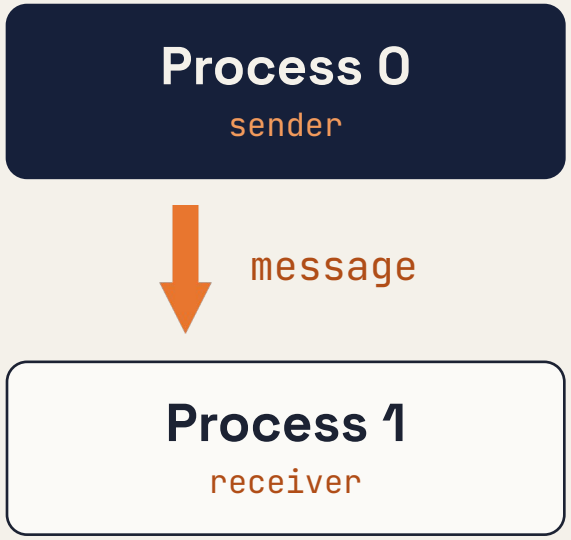
08
Practical exercises

What is message passing?

Processes communicate by **sending** and **receiving** messages — instead of directly accessing another process's memory.

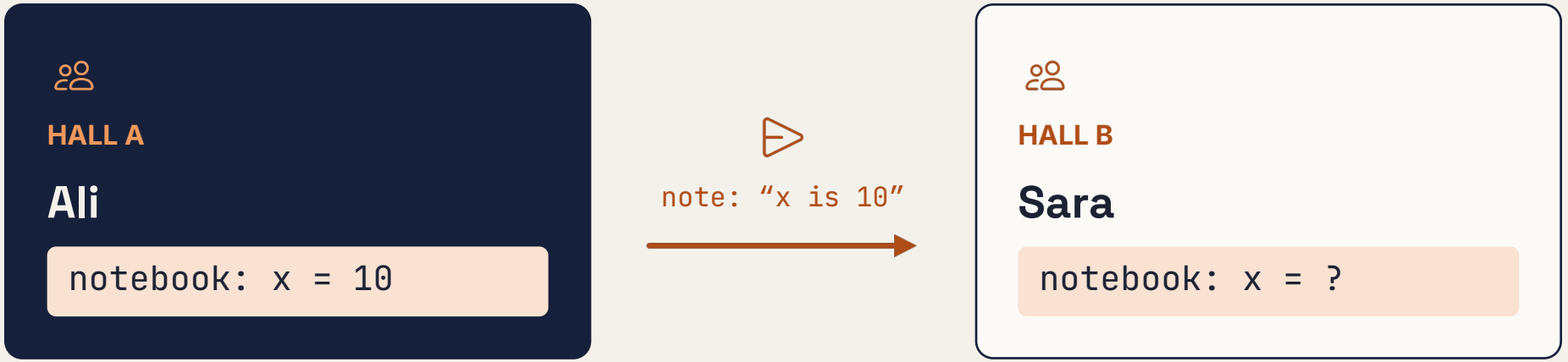
> *“I have some data. I send it to you, and you receive it.”*

A process **explicitly** sends data to another process.



EXAMPLE · MESSAGE PASSING

Two students, two exam halls



Sara can't look at Ali's notebook. The only way she learns x is if Ali **writes a note and sends it**.

Student = process	**Notebook** = local memory	**Note** = message

Why do we need message passing?



Each processor has its own local memory.

One process cannot directly access another's memory.

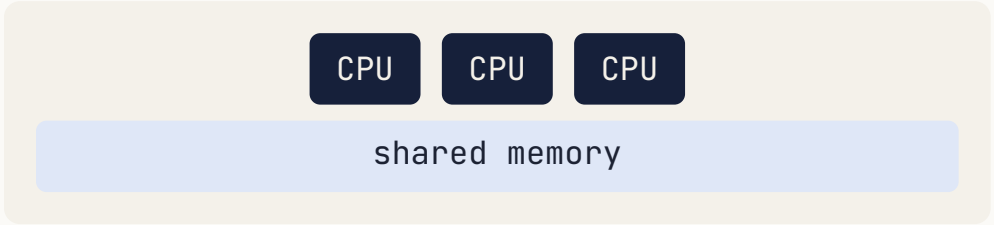
Processes need a communication mechanism.

Data is transferred through messages.

Shared memory vs distributed memory

Shared memory

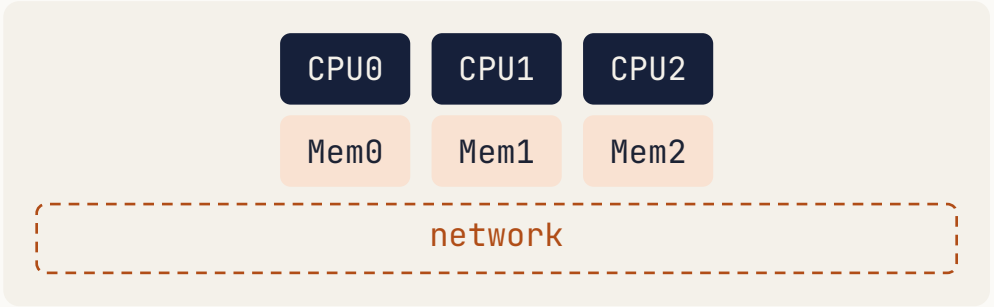
OpenMP



- Processors share memory
- Threads communicate through shared variables
- Communication is often implicit

Distributed memory

MPI



- Each processor has its own memory
- Processes communicate through messages
- Communication is explicit

EXAMPLE · SHARED VS DISTRIBUTED

One whiteboard vs separate notebooks

SHARED MEMORY · OPENMP

A study group at one whiteboard

Everyone in the same room reads and writes the same board. Sharing is instant — but two people may try to write in the same spot at once.

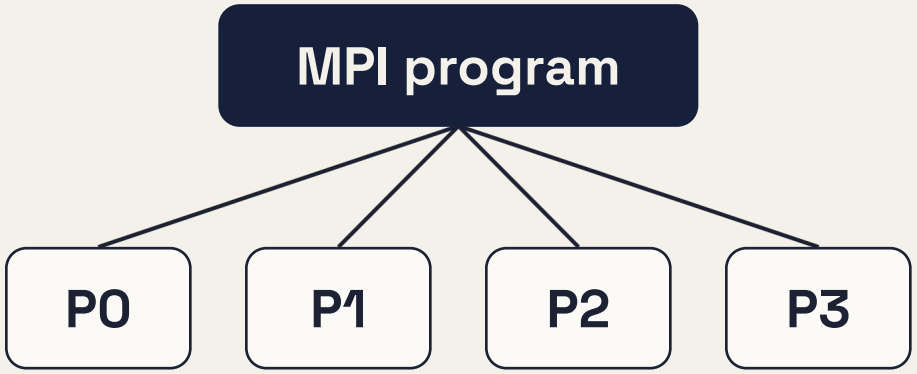
DISTRIBUTED MEMORY · MPI

A team spread across cities

Each member has their own notebook in a different city. To share a result, they must send it: an email, a WhatsApp message.

Same room, one board → threads + shared variables. Different cities, own notebooks → processes + messages.

Process-based communication



In MPI we normally work with **processes**. Each process:

- Has its own execution
- Has its own memory space
- Has a unique identity called a **rank**
- Can communicate with other processes

PART 2 · INTRODUCTION TO MPI

What is MPI?

Message Passing Interface

A **standard programming interface** for writing programs in which processes communicate by sending and receiving messages.

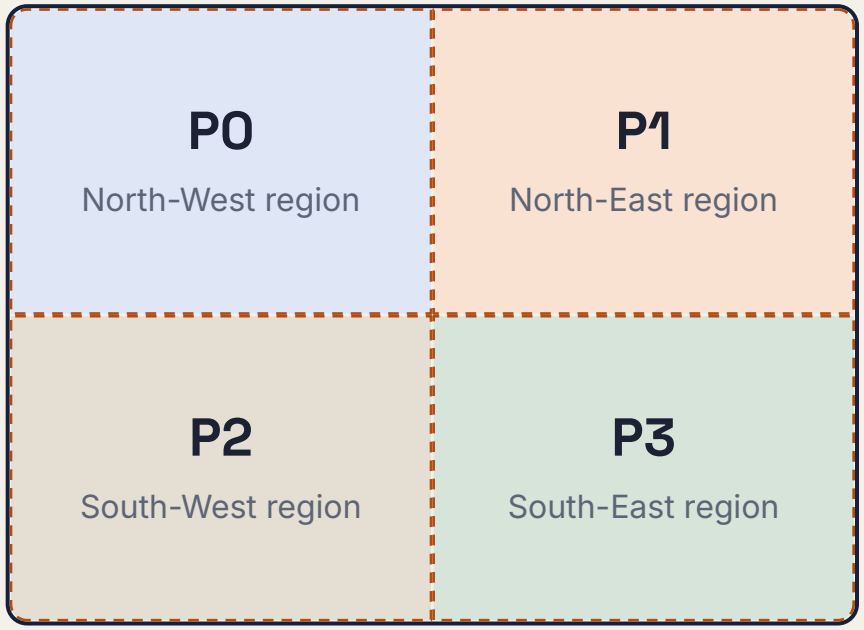
WIDELY USED IN

- Parallel computing
- Cluster computing
- Supercomputing
- Distributed-memory systems

⚠ **Important:** MPI is an interface / standard — not a single programming language. Programs call it from languages such as C, C++ and Fortran.

EXAMPLE · WHERE MPI IS USED

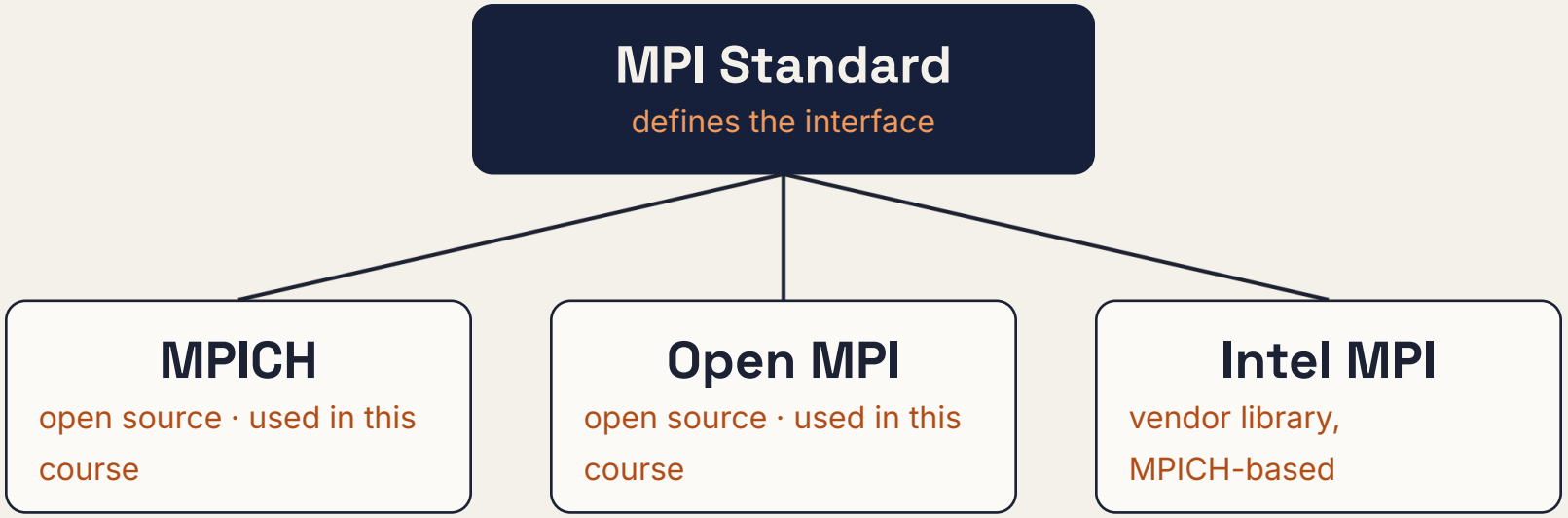
Weather forecasting



1. Split the country map into regions — one per process.
2. Each process computes the weather for its own region.
3. Wind and clouds cross borders, so neighbours **exchange boundary data** as messages (dashed lines).
4. Repeat for every time step: compute, then communicate.

Real forecasting models run this way on thousands of processes.

MPI implementations



Same standard, different software packages: a correct MPI program should run on any of them. In this course we use **MPICH** or **Open MPI**, depending on the environment.

MPI program structure

1. `MPI_Init()`
2. Get number of processes
3. Get process rank
4. Perform computation
5. Communicate
6. `MPI_Finalize()`

THE TWO MOST IMPORTANT CONCEPTS FOR BEGINNERS

Rank

Who am I?

Size

How many processes are there?

MPI rank

Every MPI process has a unique rank. Suppose we run 4 processes:

Rank 0	Rank 1	Rank 2	Rank 3
First process	Second process	Third process	Fourth process

The rank identifies the **sender** and the **receiver** of a message. Ranks always run from 0 to size − 1.

MPI_COMM_WORLD



Represents the communicator containing **all** processes in the MPI program. A communicator defines a group of processes that can communicate with one another.

EXAMPLE · RANK AND COMMUNICATOR

Roll numbers and the class group

MPI term	In a classroom
Process	A student
Rank	Roll number (starting at 0)
Size	Number of students in the class
MPI_COMM_WORLD	The whole-class WhatsApp group
Communicator	Any group whose members can message each other

“Message roll number 3 in the class group” is exactly what **MPI_Send(..., 3, ..., MPI_COMM_WORLD)** means.

PART 3 · BASIC MPI FUNCTIONS

Six functions to remember

SETUP

`MPI_Init()`

Starts MPI

SETUP

`MPI_Comm_size()`

Gets the number of processes

SETUP

`MPI_Comm_rank()`

Gets this process's rank

COMMUNICATION

`MPI_Send()`

Sends data

COMMUNICATION

`MPI_Recv()`

Receives data

TEARDOWN

`MPI_Finalize()`

Ends MPI

A basic MPI program

```c
#include <mpi.h>
#include <stdio.h>

int main(int argc, char *argv[])
{
    int rank, size;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    printf("Hello from process %d of %d\n",
           rank, size);
    MPI_Finalize();
    return 0;
}
```

1. `MPI_Init` — Start MPI
2. `MPI_Comm_rank` — Who am I?
3. `MPI_Comm_size` — How many of us?
4. `printf` — Every process prints
5. `MPI_Finalize` — Shut MPI down

Compiling and running an MPI program

1 · COMPILE

```
mpicc hello.c -o hello
```

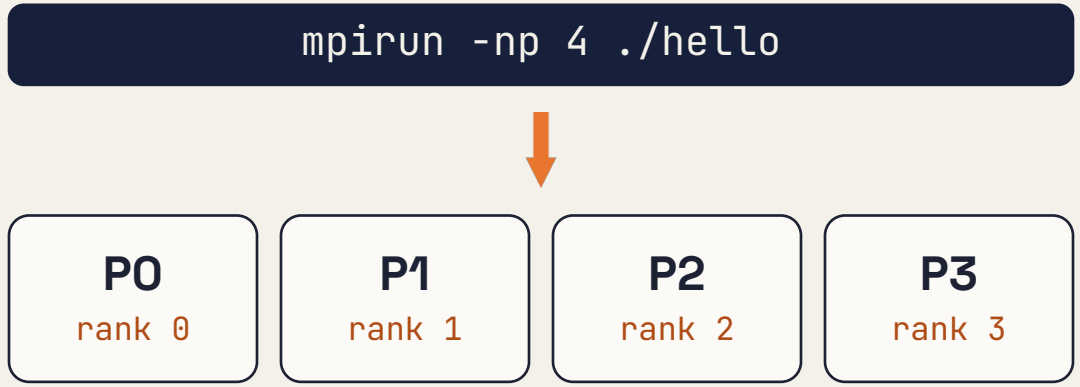
mpicc is a wrapper around the C compiler that adds the MPI library.

2 · RUN

```
mpirun -np 4 ./hello
```

-np 4 starts 4 copies (processes) of the program. Some systems use **mpiexec**.

WHAT MPIRUN DOES



Same executable, four processes, four ranks.

What happens when we run it?

```
$ mpirun -np 4 ./hello
Hello from process 0 of 4
Hello from process 1 of 4
Hello from process 2 of 4
Hello from process 3 of 4
```

Same program

Each process executes the same code.

Different rank

Each process has its own rank, so it can act differently.

This SPMD style is a form of **MIMD** parallel execution (Lecture 3).

EXAMPLE · USING THE RANK

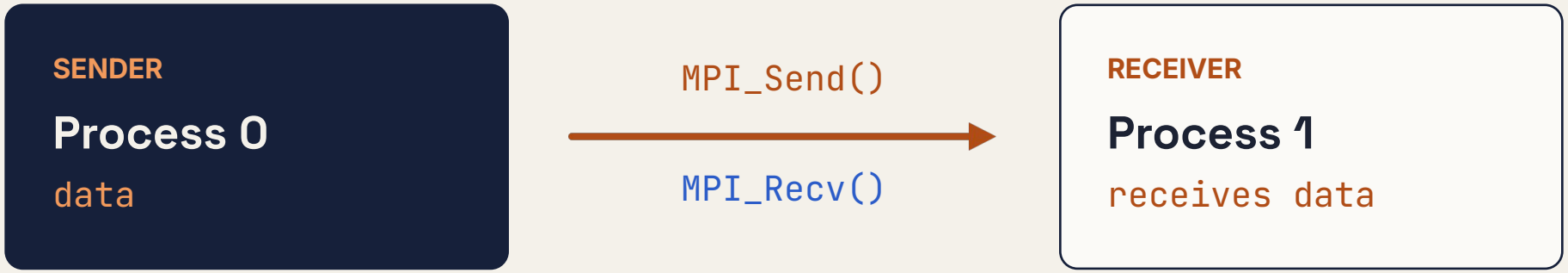
One program, different roles

```
if (rank == 0) {
    printf("I am the teacher (rank 0)\n");
} else {
    printf("I am student %d\n", rank);
}
```

Every process runs this same code. The **if** on the rank gives each process a different job.

```
$ mpirun -np 4 ./roles
I am the teacher (rank 0)
I am student 1
I am student 2
I am student 3
```

Point-to-point communication



Communication between **two** processes is called point-to-point communication: one process is the sender, the other is the receiver.

MPI_Send()

```
MPI_Send(
    data,
    count,
    datatype,
    destination,
    tag,
    communicator
);
```

Sends data from one process to another.

Parameter	Meaning
data	Address of the data to send
count	Number of elements
datatype	Type of each element, e.g. MPI_INT
destination	Rank of the receiving process
tag	Integer label for the message
communicator	Process group, e.g. MPI_COMM_WORLD

MPI_Recv()

```
MPI_Recv(
    data,
    count,
    datatype,
    source,
    tag,
    communicator,
    status
);
```

Receives data sent by another process.

Parameter	Meaning
data	Buffer that stores the received data
count	Maximum number of elements
datatype	Type of each element, e.g. MPI_INT
source	Rank of the sending process
tag	Must match the sender's tag
communicator	Process group, e.g. MPI_COMM_WORLD
status	Details about the received message

EXAMPLE · SEND AND RECEIVE

MPI_Send is like posting a letter

From: rank 0

To: **rank 1**

Subject (tag): **0**

```
1 × MPI_INT
x = 10
```

MPI_Send	Posting a letter
data	The letter's contents
count	How many pages
datatype	What language it's written in
destination	The address on the envelope
tag	The subject line
communicator	The postal service used

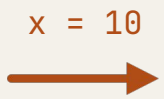
MPI_Recv is checking your mailbox for a letter from a given sender with a given subject.

Simple send / receive example

Process 0

```
int x = 10;

MPI_Send(&x, 1, MPI_INT,
         1, 0, MPI_COMM_WORLD);
```



Process 1

```
int x; MPI_Status status;
MPI_Recv(&x, 1, MPI_INT,
         0, 0, MPI_COMM_WORLD,
         &status); // x = 10
```

QUESTION

If Process 0 sends the value 10 to Process 1, where does Process 1 get the value?

ANSWER

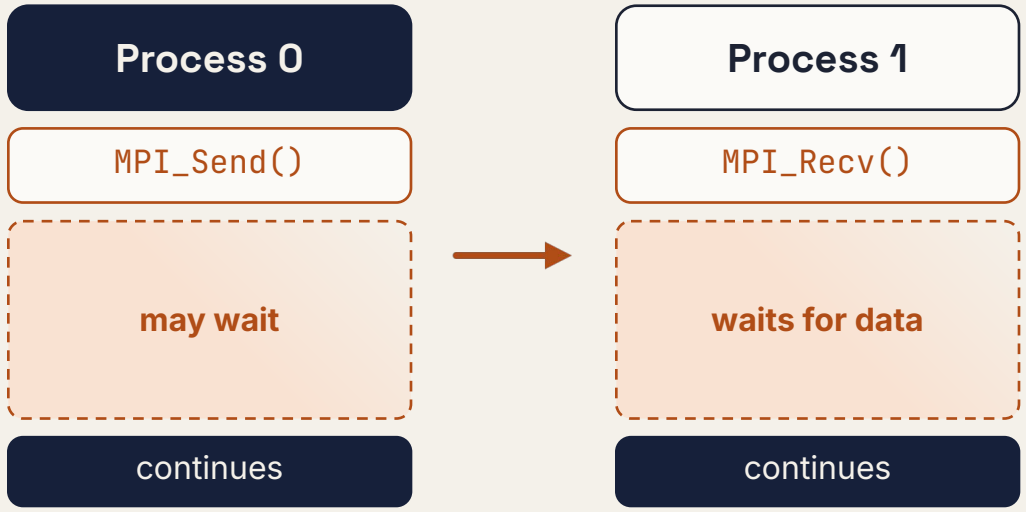
Through the **message** sent by Process 0 — not from shared memory.

Blocking communication

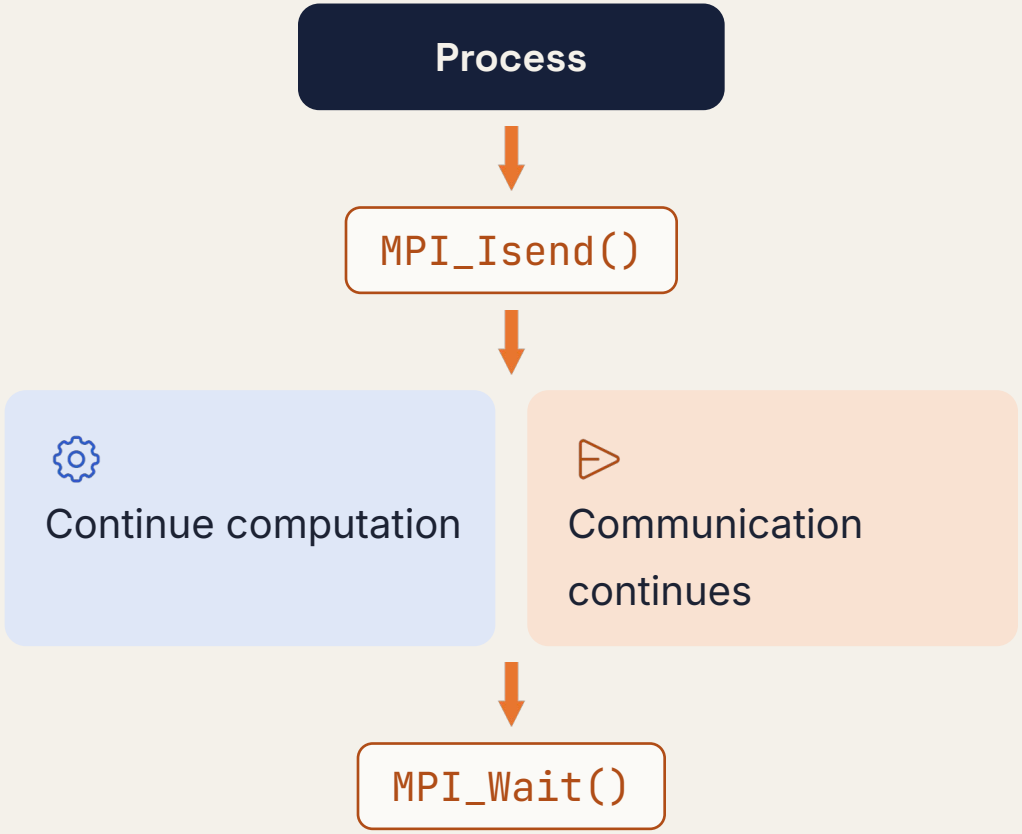
A blocking operation may **wait** until the communication can proceed — the program pauses at the call before continuing.

> *“Wait until the communication operation is completed enough to continue.”*

MPI_Send() and MPI_Recv() are blocking. For a send, “enough” means the data buffer can be safely reused.



Non-blocking communication



The process **starts** the communication, keeps doing other work, and waits only when it needs the result.

`MPI_Isend()`	start a send
`MPI_Irecv()`	start a receive
`MPI_Wait()`	wait to finish

Main idea: communication and computation can potentially overlap.

Synchronous vs asynchronous

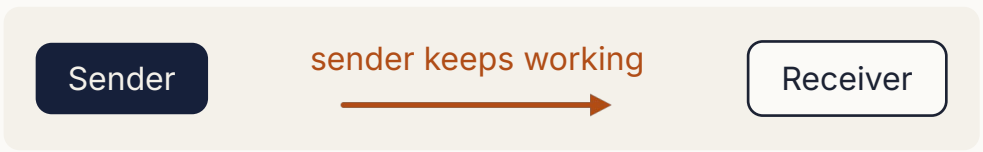
Synchronous



The communication is coordinated between the participants.

Example: MPI_Ssend() finishes only once the receiver has started receiving.

Asynchronous



The sender does not necessarily wait for the receiver before continuing.

⚠ **Important:** blocking / non-blocking and synchronous / asynchronous describe related but different aspects of communication.

EXAMPLE · COMMUNICATION MODES

Everyday versions of each mode

BLOCKING

Waiting at the shop counter

You stand there until your order is handed over, then leave.

`MPI_Send() / MPI_Recv()`

NON-BLOCKING

Ordering on a food app

Place the order, keep studying, collect it when it's ready.

`MPI_Isend() … MPI_Wait()`

SYNCHRONOUS

A phone call

Both people must be on the line at the same time.

`MPI_Ssend()`

ASYNCHRONOUS

A voice note

Record it and get on with your day; they listen later.

`buffered sends`

EXAMPLE · COMMON MISTAKE

Deadlock: everyone waits forever

PROCESS 0

```
MPI_Recv(&y, 1, MPI_INT, 1, 0,
         MPI_COMM_WORLD, &st);
MPI_Send(&x, 1, MPI_INT, 1, 0,
         MPI_COMM_WORLD);
```

⏳ waiting for P1…

PROCESS 1

```
MPI_Recv(&y, 1, MPI_INT, 0, 0,
         MPI_COMM_WORLD, &st);
MPI_Send(&x, 1, MPI_INT, 0, 0,
         MPI_COMM_WORLD);
```

⏳ waiting for P0…

WHY

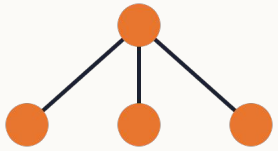
Both start by **receiving**, so nobody ever sends. Like two friends each waiting for the other to call first.

FIX

Swap the order on one side: P0 sends then receives; P1 receives then sends.

What is collective communication?

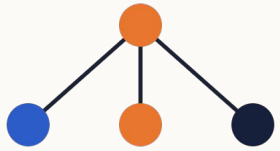
Sometimes communication involves **many processes**, not just two. MPI provides collective operations for these patterns:



ONE → ALL

`MPI_Bcast`

Same data to every process



ONE → ALL

`MPI_Scatter`

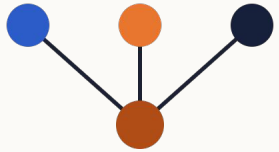
A different piece to each process



ALL → ONE

`MPI_Gather`

Collect pieces at one process



ALL → ONE

`MPI_Reduce`

Combine values into one result

EXAMPLE · COLLECTIVES

Collectives in a classroom

MPI_Bcast

The teacher announces the exam date to the whole class.

MPI_Scatter

The teacher gives each student a different chapter to summarise.

MPI_Gather

The teacher collects every student's summary.

MPI_Reduce

Marks are added up into a class total — or MPI_MAX finds the top score.

The teacher plays the role of the **root** process — usually rank 0.

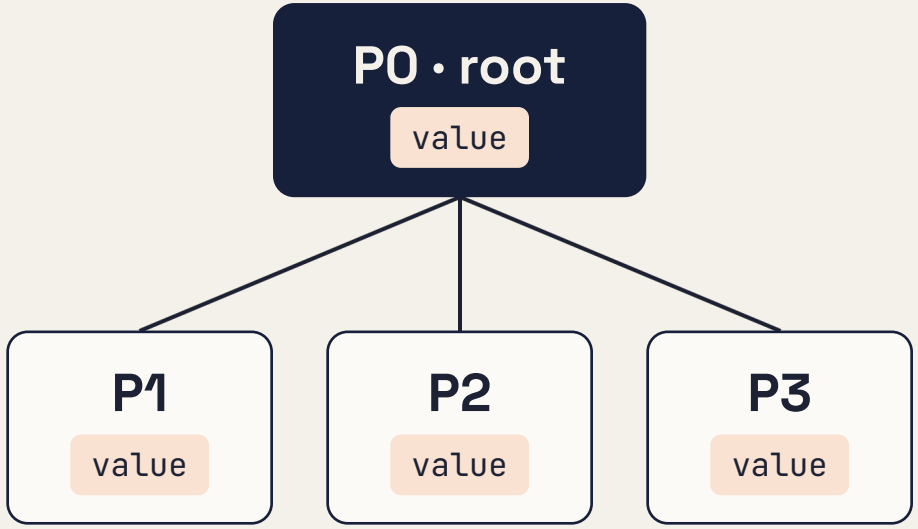
MPI_Bcast — broadcast

One process sends the **same data** to all processes.

EXAMPLE

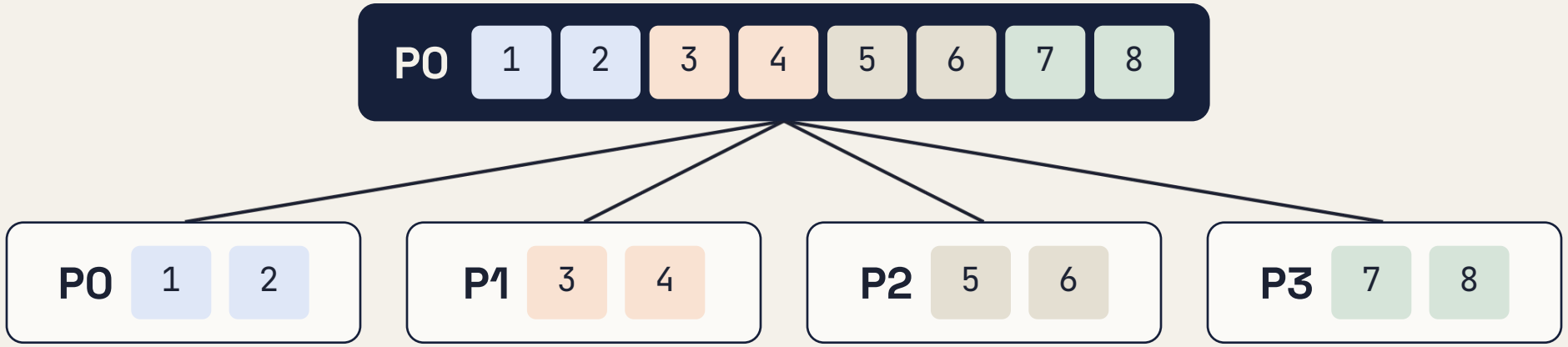
Process 0 has a value that all other processes need. MPI_Bcast() distributes that value.

```
MPI_Bcast(&value, 1, MPI_INT,
          0, MPI_COMM_WORLD);
```



MPI_Scatter

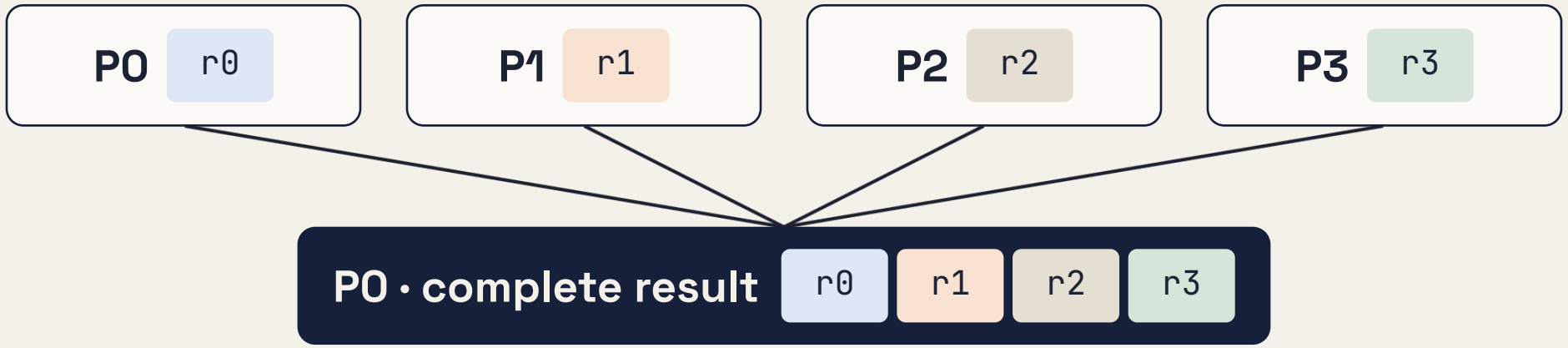
One process divides the data and sends a **different portion** to each process — very useful for data-parallel processing.



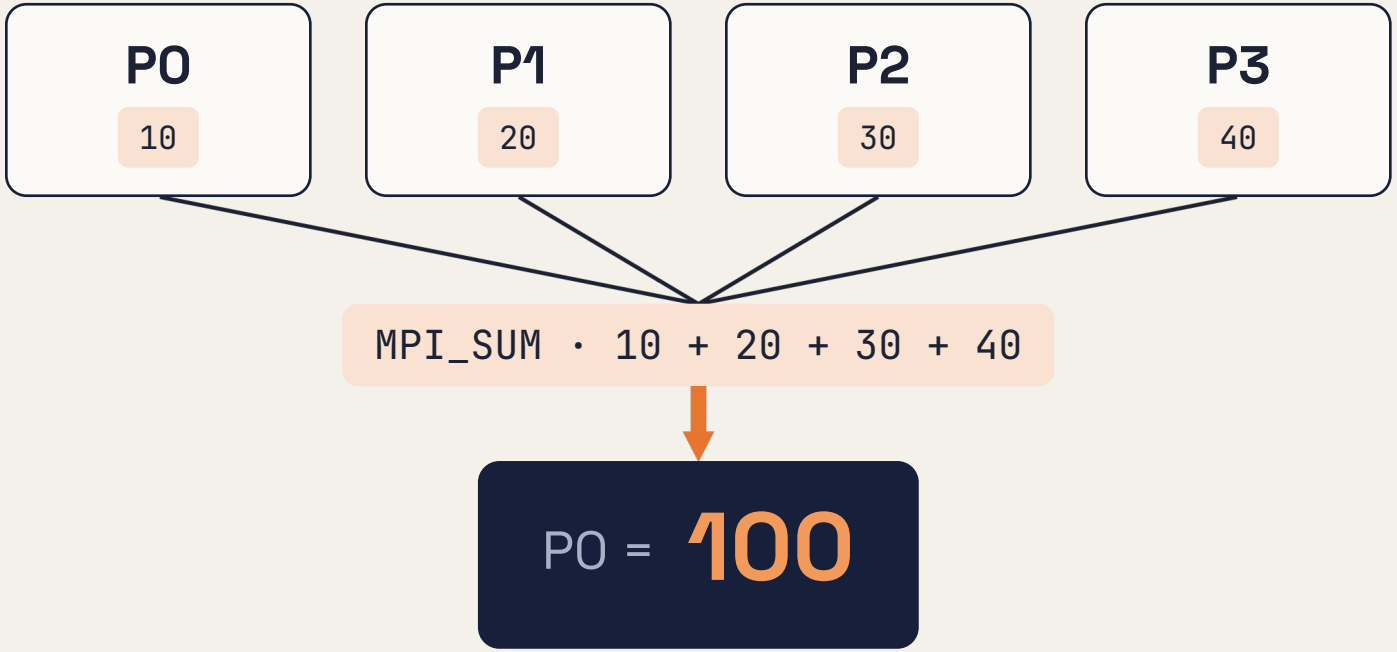
Note: the root (P0) keeps the first portion for itself.

MPI_Gather

The opposite of scatter: different processes send their **results** to one process.



MPI_Reduce



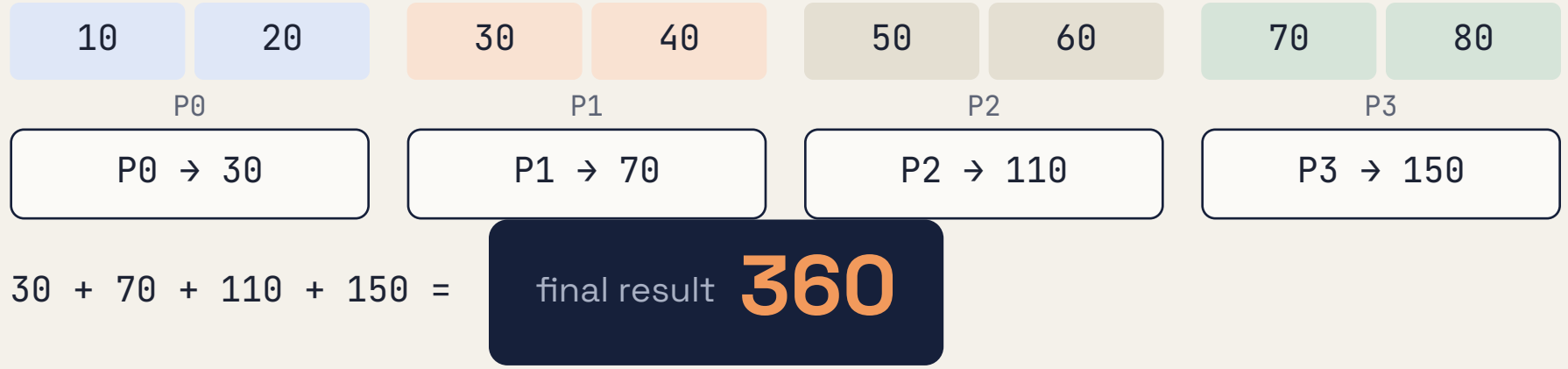
Reduce **combines** values from multiple processes into one result at the root.

POSSIBLE OPERATIONS

- `MPI_SUM`
- `MPI_MAX`
- `MPI_MIN`
- `MPI_PROD`

Example: adding an array

A = [10, 20, 30, 40, 50, 60, 70, 80] with 4 processes.



EXAMPLE · ARRAY SUM

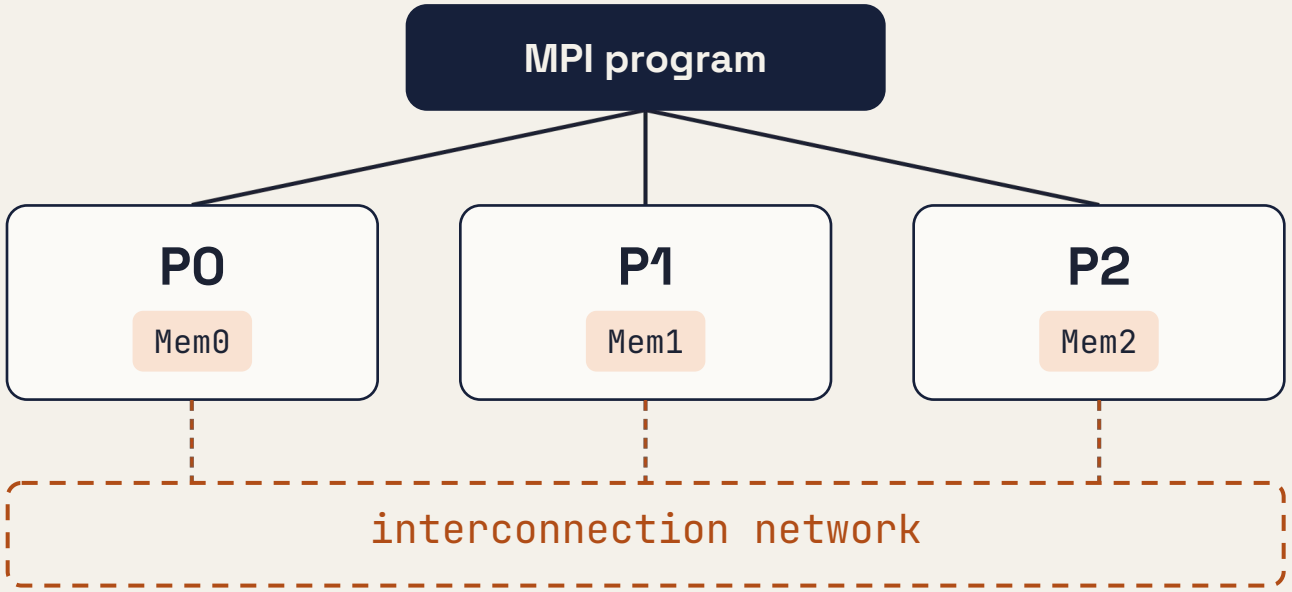
The same example in code

```
int A[8] = {10, 20, 30, 40, 50, 60, 70, 80};
int part[2], local, total;

// Step 1: give each process 2 elements
MPI_Scatter(A, 2, MPI_INT, part, 2, MPI_INT,
            0, MPI_COMM_WORLD);
// Step 2: each process adds its own part
local = part[0] + part[1];
printf("P%d partial sum = %d\n", rank, local);
// Step 3: combine partial sums on P0
MPI_Reduce(&local, &total, 1, MPI_INT, MPI_SUM,
           0, MPI_COMM_WORLD);
if (rank == 0) printf("Total = %d\n", total);
```

```
$ mpirun -np 4 ./sum
P0 partial sum = 30
P1 partial sum = 70
P2 partial sum = 110
P3 partial sum = 150
Total = 360
```

The MPI communication model



EACH PROCESS HAS

- Its own memory
- Its own execution
- Its own rank

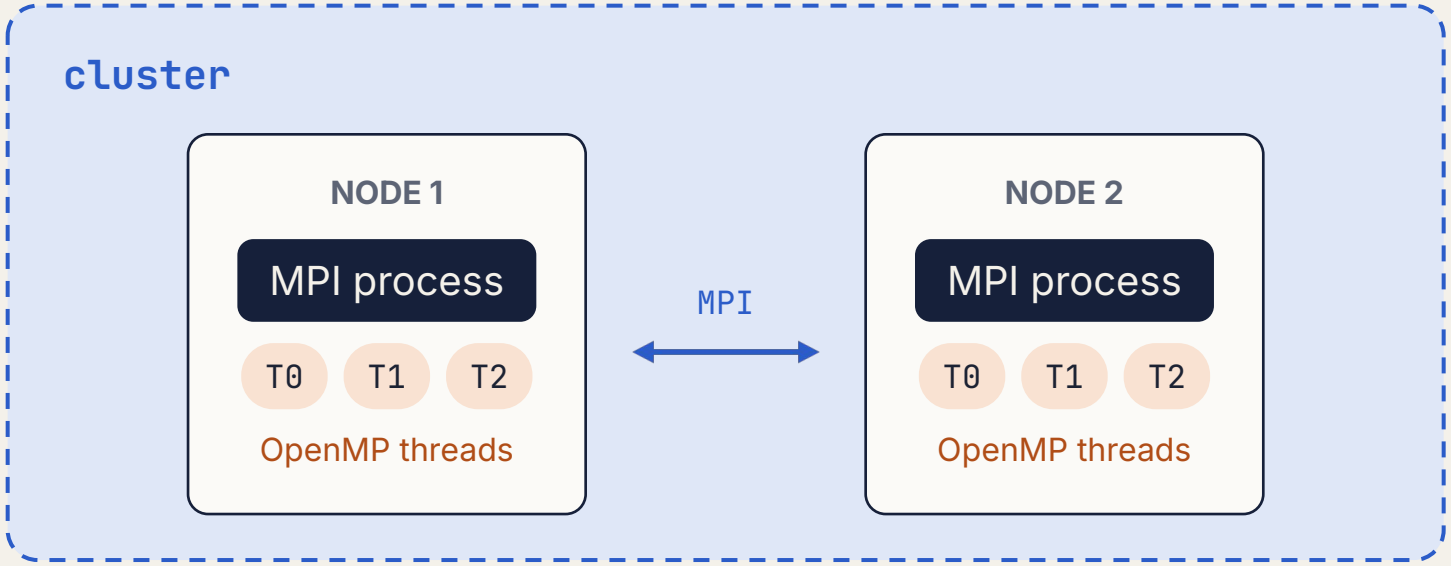
MPI provides the communication mechanism.

OpenMP vs MPI

	OpenMP	MPI
Memory	Shared memory	Distributed memory
Workers	Threads	Processes
Exchange data via	Shared variables	Messages
Hardware	Usually one shared-memory system	Can use multiple machines
Communication	Easy, implicit	Explicit
Syntax	#pragma omp	MPI functions
Example	Multicore CPU	Cluster

Recall Lecture 4b: #pragma omp parallel creates a team of threads executing the parallel region.

Hybrid programming



Modern systems can combine MPI and OpenMP.

`MPI`
Communication **between** nodes

`OpenMP`
Parallelism **inside** a node

EXAMPLE · HYBRID PROGRAMMING

A company with offices in two cities

Each office

is one **node** (machine) in the cluster.

Inside an office

staff share one whiteboard and split the work → **OpenMP threads**.

Between offices

they can't see each other's board, so they email results → **MPI messages**.

Big supercomputers work exactly like this: MPI across nodes, OpenMP (or GPUs) inside each node.

Exercise 1 — send and receive

Write an MPI program that:

1. Creates 4 processes.
2. Displays the rank of each process.
3. Process 0 sends the value 100 to Process 1.
4. Process 1 receives the value.
5. Process 1 displays the received value.

EXPECTED OUTPUT

```
Process 0: Sending 100
Process 1: Received 100
```

Exercise 1 — one possible solution

```c
#include <mpi.h>
#include <stdio.h>
int main(int argc, char *argv[]) {
    int rank, value;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    printf("Process %d is running\n", rank);
    if (rank == 0) {
        value = 100;
        printf("Process 0: Sending %d\n", value);
        MPI_Send(&value, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
    } else if (rank == 1) {
        MPI_Recv(&value, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        printf("Process 1: Received %d\n", value);
    }
    MPI_Finalize();
    return 0;
}
```

Exercise 2 — reduce

P0	P1	P2	P3
10	20	30	40

Use **MPI_Reduce()** with **MPI_SUM**. What will be the final result?

ANSWER

10 + 20 + 30 + 40 = 100

REVIEW

Check your understanding

Q1

Can process 1 directly read a variable in process 0's memory?

No — process 0 must send it in a message.

Q2

You run mpirun -np 6. Which ranks exist?

Ranks 0 to 5; size = 6.

Q3

Which collective gives each process a different piece of an array?

MPI_Scatter.

Q4

Both processes call MPI_Recv first, then MPI_Send. What happens?

Deadlock — both wait forever.

Q5

How do you find the highest mark held across all processes?

MPI_Reduce with MPI_MAX.

Q6

A cluster of 10 machines: OpenMP or MPI?

MPI between machines (OpenMP can help inside each).

SUMMARY

Common MPI terms

Process
An independent execution unit.

Rank
Unique ID of an MPI process.

Communicator
Defines a group of processes that can communicate.

Message
Data transferred between processes.

Sender
Process that sends data.

Receiver
Process that receives data.

Point-to-point
Communication between two processes.

Collective
Communication involving a group of processes.

SUMMARY

Key takeaways

01

Distributed memory

Processes can't see each other's memory, so they exchange messages.

02

MPI

A standard interface; MPICH and Open MPI implement it.

03

Rank & size

Who am I? How many are we? Every program branches on rank.

04

Point-to-point

MPI_Send / MPI_Recv; watch out for deadlock.

05

Collectives

Bcast, Scatter, Gather, Reduce for group communication.

06

Hybrid

MPI between nodes, OpenMP inside a node.

PRACTICE

Lab tasks / homework

1. **Ring:** pass a token from rank 0 → 1 → 2 → … → back to 0. Each process prints what it received.
2. **Sum 1 to 1000:** split the range across processes, compute partial sums, combine with MPI_Reduce.
3. **Maximum mark:** each process holds one mark; find the highest using MPI_Reduce with MPI_MAX.
4. **Short answer:** explain why a program where both processes call MPI_Recv first deadlocks, and fix it.

Run each program with 2, 4 and 8 processes and check the output.

FURTHER READING

References

MPI Forum	The official MPI standard documents — mpi-forum.org
P. Pacheco	An Introduction to Parallel Programming — chapter on MPI
W. Gropp, E. Lusk, A. Skjellum	Using MPI: Portable Parallel Programming with the Message-Passing Interface
MPI Tutorial	Beginner tutorials with examples — mpitutorial.com

Questions?

Next step: write, compile and run your first MPI program in the lab.

```
mpirun -np 4 ./hello
```